

# Infosys SP / DSE Preparation Guide

---

Specialist Programmer & Digital Specialist Engineer — 2026 Batch

Compiled for Apoorv

July 2026

- [INFOSYS-SP-DSE-PREPARATION](#)
  - [Structure](#)
  - [How to use this repo](#)
  - [Status](#)
  - [Source note](#)
- [Overview: Infosys SP and DSE Roles](#)
  - [What are these roles?](#)
  - [SP vs DSE — how to think about the difference](#)
  - [Salary bands](#)
  - [Hiring stages \(typical flow\)](#)
  - [Key prep principle](#)
- [Syllabus and Exam Pattern](#)
  - [Coding round structure](#)
  - [Round-by-round flow](#)
  - [What differs between SP and DSE at the syllabus level](#)
  - [Exam logistics notes](#)
  - [Prep-time allocation \(recommended\)](#)
- [Coding Round — Full Topic Checklist](#)
  - [Easy \(Q1-tier\)](#)
  - [Medium \(Q2-tier\)](#)
  - [Hard \(Q3-tier\)](#)
  - [Cross-cutting skills \(test throughout, not a separate section\)](#)
  - [DSE-specific additions](#)
- [Previous-Year Question Patterns](#)
  - [Pattern 1 — Modular arithmetic / number theory framing](#)
  - [Pattern 2 — “Rearrange array into two groups with minimum adjacent swaps”](#)
  - [Pattern 3 — Probability / simulation with expected value](#)
  - [Pattern 4 — XOR-maximization / selection under a size constraint](#)
  - [Pattern 5 — Base conversion](#)
  - [General notes on the question bank](#)
- [Technical Interview + HR Prep](#)
  - [Technical interview — question bank by area](#)
  - [HR / behavioral round](#)
  - [Interview-day logistics](#)
- [External Resources](#)
  - [Written guides](#)
  - [Code / practice repos](#)

- [Video](#)
- [Caveats](#)

# INFOSYS-SP-DSE-PREPARATION

Structured, self-contained prep material for the Infosys **Specialist Programmer (SP)** and **Digital Specialist Engineer (DSE)** hiring process — 2026 batch.

This repo is built so that no supplementary source is required mid-prep: syllabus, coding-round topic breakdown, previous-year-question patterns, and interview prep are all here in full.

## Structure

| Folder                                      | Contents                                                                                             |
|---------------------------------------------|------------------------------------------------------------------------------------------------------|
| <a href="#">01-overview/</a>                | What SP and DSE actually are, how they differ, salary bands, hiring stages                           |
| <a href="#">02-syllabus-and-pattern/</a>    | Exam pattern, timing, cutoffs, round-by-round breakdown                                              |
| <a href="#">03-coding-topics/</a>           | Full topic checklist by difficulty (Easy / Medium / Hard) with what to master in each                |
| <a href="#">04-previous-year-questions/</a> | Previous-year coding question patterns, paraphrased with approach notes (not verbatim reproductions) |
| <a href="#">05-interview-prep/</a>          | Technical interview question bank (DSA, OOP, SQL, system design, resume-based) + HR round prep       |
| <a href="#">06-resources/</a>               | External sources used to compile this repo, plus a GitHub repo and YouTube playlist worth following  |

## How to use this repo

1. Start with [01-overview](#) and [02-syllabus-and-pattern](#) — 15 minutes, gives you the shape of the whole process.
2. Work [03-coding-topics](#) top to bottom, Easy → Hard. Don't skip to Hard early; the coding round always includes at least one Easy and one Medium question.
3. Use [04-previous-year-questions](#) for pattern recognition, not memorization — Infosys reuses *patterns*, not exact problems.
4. Once you're clearing mock coding rounds consistently, move to [05-interview-prep](#).

## Status

☐ Coding topics — practiced Easy

- ☐ Coding topics — practiced Medium
- ☐ Coding topics — practiced Hard
- ☐ Previous-year patterns — reviewed
- ☐ Interview prep — DSA/OOP/SQL
- ☐ Interview prep — resume Q&A rehearsed
- ☐ Mock interview done

## Source note

This material was compiled from publicly available prep guides (PrepInsta, GreatLearning, TalentBattle, OnlineStudy4U, GeeksforGeeks) as of July 2026 — see `06-resources/sources.md` for the full list and links. Content has been paraphrased and reorganized, not copied verbatim; always cross-check exam pattern details against your official Infosys hiring email, since rounds can shift batch to batch. -e

ewpage

# Overview: Infosys SP and DSE Roles

## What are these roles?

Infosys runs several distinct hiring tracks beyond the standard entry-level System Engineer (SE) role. Two of the more technical, higher-paying tracks are:

- **Specialist Programmer (SP)** — a pure coding/algorithms role. Candidates are evaluated heavily on DSA and algorithm design. Internally, SP has sub-levels (L1/L2/L3) with increasing scope and pay.
- **Digital Specialist Engineer (DSE)** — a full-stack/digital role covering modern application development, automation, analytics, and cloud-adjacent skills. Coding difficulty is generally a notch below SP, but breadth across the stack (frontend, backend, APIs, databases) matters more.

Both sit above the standard ASE (Associate Software Engineer) track and below/parallel to Infosys's "Power Programmer" hiring line.

## SP vs DSE — how to think about the difference

| Aspect             | SP                                                      | DSE                                                             |
|--------------------|---------------------------------------------------------|-----------------------------------------------------------------|
| Core focus         | DSA, algorithms, competitive-programming-style problems | Full-stack development, digital tech, applied engineering       |
| Coding difficulty  | Higher — graph/DP-heavy problems appear more often      | Moderate — more application-style problems                      |
| Interview emphasis | Time/space complexity, algorithmic tradeoffs            | Resume projects, frameworks used, API design, DB schema choices |
| Typical fit        | Strong competitive programmers, DSA-first prep          | Strong project builders, full-stack exposure                    |

## Salary bands

Reported packages for freshers across SP levels and DSE run roughly **₹10-21 LPA**, with SP L2/L3 typically at the higher end. Exact figures vary by batch and location — treat these as directional, not guaranteed.

## Hiring stages (typical flow)

1. **Registration / eligibility check** — branch, CGPA, backlog criteria as per the official notification.
2. **Online coding assessment** — the core filter. 3 coding questions, 3 hours total, different topics and difficulty per question (see [02-syllabus-and-pattern/](#) ).
3. **Technical interview** — DSA, OOP, SQL, and deep dives into your resume/projects.
4. **HR / behavioral round** — culture fit, situational judgment, “why Infosys,” career goals.
5. **Offer** — role and level (SP L1/L2/L3 or DSE) assigned based on overall performance, not always known in advance.

## Key prep principle

Across every source used to build this repo, one point repeats: **the online coding assessment is the hardest filter**. If you clear it, the interview stage is very manageable for anyone with solid fundamentals. This means coding-round prep (Section 03) should get the majority of your prep time, not the interview-question banks. -e

ewpage

## Syllabus and Exam Pattern

---

### Coding round structure

- **3 coding questions, 3 hours total.**
- Each question is drawn from a **different topic** and pitched at a **different difficulty level** — the round is deliberately spread, not three problems of the same type.
- Typical split reported across sources:
  - **Q1 — Easy:** arrays/strings, basic data structures (stacks, queues, hash maps, linked lists). Tests basic logic-building and correctness.
  - **Q2 — Medium:** greedy algorithms, recursion, backtracking, or optimization-style problems requiring some analytical thinking.
  - **Q3 — Hard:** dynamic programming or graph-based problems, testing advanced algorithmic skill.
- **Passing all test cases is not mandatory** for every question — partial credit exists — but there can be **sectional/topical cutoffs**, meaning you can’t skip a section entirely and rely on the other two carrying you. Attempt all three.
- Time management matters more than raw difficulty: with 3 hours for 3 questions, a common strategy is ~45-50 minutes on Q1, ~60 on Q2, and the remainder on Q3, leaving buffer for edge-case testing.

### Round-by-round flow

1. **Online coding assessment** (see above) — the primary filter.

2. **Technical interview** — if you clear the coding round, this is comparable in difficulty to any solid campus technical interview: DSA fundamentals, OOP, SQL, and resume/project deep-dives.
3. **HR round** — standard behavioral/fit round.

## What differs between SP and DSE at the syllabus level

- **SP**: heavier weight on algorithmic questions (DP, graphs) even at the “coding round” stage; interview probes deeper into complexity analysis and algorithm choice tradeoffs.
- **DSE**: coding round difficulty is moderate; interview leans into full-stack/digital skills — frameworks, REST APIs, cloud/automation concepts, and how you built your projects, not just algorithm design.

## Exam logistics notes

- Infosys hiring (on-campus or off-campus/HackWithInfy-style drives) does **not run on a fixed public schedule** — dates depend on organizational hiring needs, so always confirm timing through your registered email/campus placement cell rather than assuming a fixed annual calendar.
- Eligibility criteria (branch, CGPA, backlog rules) are set per drive — check the official notification for your batch rather than relying on last year’s numbers.

## Prep-time allocation (recommended)

Given that the coding round is consistently flagged as the harder filter:

- ~**65-70%** of prep time → coding topics + previous-year pattern practice (Sections 03-04)
- ~**25-30%** of prep time → technical interview prep: DSA/OOP/SQL revision, resume Q&A (Section 05)
- ~**5%** → HR/behavioral prep -e

ewpage

## Coding Round — Full Topic Checklist

---

Organized by the same Easy / Medium / Hard split the actual coding round uses, so you can practice in the order you’ll be tested.

### Easy (Q1-tier)

Focus: correctness and clean logic-building, not cleverness.

- ☐ Arrays — traversal, rotation, prefix sums, two-pointer basics, sliding window basics
- ☐ Strings — palindrome checks, anagram checks, string reversal/manipulation, pattern matching basics
- ☐ Stacks — balanced parentheses, next-greater-element, basic expression evaluation
- ☐ Queues — circular queue, basic simulation problems
- ☐ Hash maps — frequency counting, two-sum style lookups
- ☐ Linked lists — reversal, cycle detection, merging two sorted lists, basic insert/delete

## Medium (Q2-tier)

Focus: choosing the right technique, not just brute force.

- ☐ Greedy algorithms — interval scheduling, activity selection, coin-change (greedy variant), minimum-swaps-style problems
- ☐ Recursion — generating subsets/permutations, divide-and-conquer basics
- ☐ Backtracking — N-Queens style, combination/subset generation with constraints, Sudoku-style constraint problems
- ☐ Optimization problems — problems with a “minimize/maximize X operations” framing (a recurring Infosys phrasing pattern — see Section 04)
- ☐ Trees — binary tree traversals (in/pre/post-order), BST operations, height/diameter problems

## Hard (Q3-tier)

Focus: recognizing the underlying pattern under time pressure.

- ☐ Dynamic programming — 1D DP (climbing stairs, house robber style), 2D DP (grid paths, edit distance), knapsack variants, subsequence problems (LCS, LIS)
- ☐ Graphs — BFS/DFS traversal, connected components, shortest path (Dijkstra/BFS on unweighted graphs), cycle detection, topological sort
- ☐ Bit manipulation — XOR-based problems (a recurring Infosys pattern — see Section 04), bitmasking for subset problems
- ☐ Number theory basics — modular arithmetic, GCD/LCM, base conversion problems (also a recurring pattern)

## Cross-cutting skills (test throughout, not a separate section)

- ☐ Time and space complexity analysis — be ready to state Big-O for every solution you write, unprompted
- ☐ Writing edge-case-safe code (empty input, single element, all-same-elements, overflow-prone inputs)
- ☐ Clean variable naming and function decomposition — interviewers do look at code quality, not just test-case pass rate

## DSE-specific additions

If you're targeting DSE specifically, layer these on top of the DSA checklist above:

- ☐ REST API basics — writing a GET/POST endpoint in your primary framework (a real recent DSE interview question involved writing basic FastAPI endpoints)
- ☐ SQL — joins, aggregations, subqueries, window functions (see your `sql-notes-repo` for the deep dive)
- ☐ OOP fundamentals — the four pillars, applied to a real design question, not just definitions
- ☐ Git workflow — be ready to describe your actual workflow (branching, merging, resolving conflicts), not just name the commands -e

ewpage

## Previous-Year Question Patterns

---

Important: these are **paraphrased patterns and approach notes**, not copy-pasted problem statements — treat them as “what shape of problem to expect,” then go practice the underlying pattern on LeetCode/GfG rather than memorizing an exact statement. Infosys reuses *patterns* year to year, not verbatim questions.

## Pattern 1 — Modular arithmetic / number theory framing

**Shape:** You’re given a number  $N$  and asked to find some property of a subset of integers in a range, expressed under modular arithmetic (e.g., “find the largest subset whose product is  $1 \bmod N$ ”).

**Underlying skill tested:** modular exponentiation, multiplicative order, basic number theory — not raw brute force.

**How to practice:** GfG’s “Modular Arithmetic” section; LeetCode problems tagged `math` + `number-theory`.

## Pattern 2 — “Rearrange array into two groups with minimum adjacent swaps”

**Shape:** Given an array, rearrange it so all elements satisfying some condition (even/odd, positive/negative, above/below a threshold) end up grouped together — using the *minimum number of adjacent swaps*.

**Underlying skill tested:** two-pointer technique, greedy counting (you usually don’t need to simulate the swaps — you can count inversions/mismatches directly).

**How to practice:** LeetCode “minimum swaps to group all 1s together” and its variants.

## Pattern 3 — Probability / simulation with expected value

**Shape:** A probability question involving drawing items (balls, cards) from a container, asking for the probability of some final-state event across multiple categories.

**Underlying skill tested:** basic combinatorics, expected value reasoning, careful precision handling (answers often allow small absolute/relative error tolerance — read the output-format constraints carefully).

**How to practice:** GfG “probability and expected value” problems; pay special attention to how the problem defines acceptable error tolerance in the output.

## Pattern 4 — XOR-maximization / selection under a size constraint

**Shape:** Given an array, choose at most  $N/2$  elements to maximize (or achieve a target) XOR of the chosen subset.

**Underlying skill tested:** bitmasking, sometimes a greedy bit-by-bit construction (decide each bit from the most significant down), or DP over subsets for smaller constraints.

**How to practice:** LeetCode/GfG problems tagged `bit-manipulation` + `subset`; specifically look at “maximum XOR subset” style problems.

## Pattern 5 — Base conversion

**Shape:** Convert a number from decimal into a different base `K`, sometimes framed indirectly (e.g., describing a positional numeral system and asking you to reconstruct the representation).

**Underlying skill tested:** careful implementation of repeated division/remainder logic, off-by-one handling for the leading digit.

**How to practice:** straightforward — implement base conversion for a few different bases from scratch without a library function, so you're fast under time pressure.

## General notes on the question bank

- Across previous years, at least one question per set tends to be a straightforward Easy/Medium DSA problem (arrays, strings, stacks/queues) — don't over-index prep time on the exotic patterns above at the expense of fundamentals.
- Questions are pulled from a large rotating bank — expect the *style* above, not these exact five problems.
- Where to go deeper: the GitHub repo `karthikreddy-7/Infosys-SP-Coding-Questions` and Preplnsta's coding-questions page both maintain larger, regularly updated banks of these patterns with worked solutions — see `06-resources/sources.md` for links. -e

ewpage

# Technical Interview + HR Prep

---

## Technical interview — question bank by area

### DSA (asked of both SP and DSE candidates)

- Explain the time/space complexity of the solution you just wrote — be ready to justify it, not just state it.
- Given a problem, walk through why you chose one data structure over another (e.g., “why a hash map here and not a sorted array”).
- Follow-up optimization questions: “can you do this in  $O(n)$  instead of  $O(n \log n)$ ?”

### OOP

- The four pillars (encapsulation, abstraction, inheritance, polymorphism) — but expect them applied to a scenario, not recited as definitions.
- Design a small class hierarchy for a given scenario (common: vehicle types, employee types, shape hierarchies) and justify your design choices.

### SQL

- Joins (inner/left/right/full), aggregations with GROUP BY/HAVING, subqueries vs. joins tradeoffs.
- Window functions if you're going for DSE — increasingly common in recent interviews.
- Cross-reference your `sql-notes-repo` for the deeper interview-question set you already built there.

### System design (lighter weight, mostly for DSE)

- Basic API design: “design a simple endpoint for X” — expect to sketch request/response shape, not a full distributed system.
- Database schema design for a small, familiar domain (e.g., a library system, an e-commerce cart).

## Resume-based (very heavily weighted — treat as equal priority to DSA)

Go through **every project and skill on your resume** and prepare answers for: - Walk me through your final year project, end to end. - What was the most challenging problem you solved in it? - Which technology did you use for the backend/frontend, and why that over the alternatives? - What’s the time complexity of the key algorithm in your project? - If you had to redesign this project from scratch, what would you change? - For any framework listed on your resume (e.g., FastAPI): be ready to write a basic GET and POST endpoint live — this is a real, recently-asked DSE interview question. - What version control system did you use, and describe your actual Git workflow (branching strategy, how you handle merge conflicts) — not just command definitions.

**Rule of thumb:** know every bullet on your resume well enough that you could be asked to defend it for five minutes straight.

## HR / behavioral round

- “Why Infosys” — have a genuine, specific answer (not generic “great company culture”).
- Standard behavioral prompts: a time you disagreed with a teammate, a time you missed a deadline, how you handle ambiguous requirements.
- Situational judgment style questions — if you want the same style of prep you used for TCS IPA’s BizSkills section, treat this the same way: reason through the most “professionally sound” response, not necessarily the most literal one.
- Know your resume’s timeline cold: gaps, certifications, internship dates — HR rounds often probe consistency.

## Interview-day logistics

- Environment: confirm whether the technical round is virtual (have your IDE/compiler ready, test your camera/mic) or in-person.
- Have 2–3 thoughtful questions ready to ask the interviewer at the end — about the specific team/role, not generic questions answerable from the company website. -e

ewpage

## External Resources

---

This repo is meant to be self-contained, but these are the primary sources it was compiled from, plus a couple of extras worth following for continuously updated content (Infosys refreshes its question banks over time).

## Written guides

- **PrepInsta — Infosys SP and DSE hub:** syllabus, placement papers, coding questions, updated per batch. <https://prepinsta.com/infosys-sp-and-dse/>
- **PrepInsta — coding questions specifically:** <https://prepinsta.com/infosys-sp-and-dse/coding-questions/>

- **GreatLearning — “How to Crack Infosys SP & DSE Interview 2026”**: the most complete single interview guide found, covers DSA/OOP/SQL/system design/resume questions plus real recent interview questions. <https://www.mygreatlearning.com/blog/infosys-sp-dse-interview-guide-2026/>
- **GreatLearning — “Top Infosys SP and DSE Coding Questions With Answers (2026)”**: <https://www.mygreatlearning.com/blog/infosys-sp-dse-coding-questions-with-answers-2026/>
- **TalentBattle — Infosys SP DSE Previous Year Questions**: actual past problem statements with explanations. <https://talentbattle.in/company-specific-previous-year-questions/infosys-sp-dse-previous-year-questions>
- **Lets-Code.co.in — 25+ previous year coding questions**: <https://www.lets-code.co.in/previousyearcodingquestion/infosys-sp-and-dse-previous-year-coding-questions/>
- **OnlineStudy4U — 100-question master sheet (Easy/Medium/Hard/Complex)**: <https://onlinestudy4u.in/infosys-dse-sp-exam-important-questions/>
- **GeeksforGeeks — Infosys SDE Sheet**: structured problem sheet with practice links. <https://www.geeksforgeeks.org/dsa/infosys-sde-sheet-interview-questions-and-answers/>
- **PlacementPreparation.io — Infosys overview**: hiring stages, mock tests. <https://www.placementpreparation.io/infosys/>

## Code / practice repos

- **GitHub — karthikreddy-7/Infosys-SP-Coding-Questions**: a repo specifically collecting Infosys SP coding questions. <https://github.com/karthikreddy-7/Infosys-SP-Coding-Questions>

## Video

- **YouTube — “Infosys 2026 Preparation | PYQ, Aptitude, Reasoning, Coding, DSE, SP, SE Roles”** (playlist): <https://www.youtube.com/playlist?list=PLFJqorivpX8xCE2ArG5JGjhLBXUqQY0oY>
- **YouTube — “Infosys SP/DSE Round 2 Preparation Strategy 2026”**: <https://www.youtube.com/watch?v=1xM5du6OZAc>
- **YouTube — “Infosys SP and DSE Coding Preparation | Previous Year Questions”** (playlist): <https://www.youtube.com/playlist?list=PLKtofb3HgEyxHTpgE5lvKHrwWihIEzhHc>

## Caveats

- Exam pattern and eligibility criteria are **not on a fixed public schedule** — Infosys hiring depends on organizational needs batch to batch. Treat everything in `02-syllabus-and-pattern/` as the most recently reported pattern, and reconfirm against your own official hiring communication.
- Salary figures are self-reported/aggregated across sources and will vary by drive, location, and level.